OWASP Top 10

the same handful of mistakes, everywhere

Watch the 5-min video

In one line: the OWASP Top 10 lists the most critical web application risks from real data; most breaches chain two or three of them, and the fixes share one pattern: enforce on the server, keep data apart from code, and be secure by default.

1 Same mistakes, thousands of apps

Most breaches don't use clever new attacks. They reuse a handful of mistakes. The OWASP Top 10 (current widely used edition: 2021) ranks web application risk categories using data from hundreds of thousands of applications plus a community survey. Know it, and you can review any design with an attacker's eyes.

2021 category	Typical example	Main fix
A01 Broken Access Control	change an ID, read someone else's record	server-side checks, deny by default (lesson 39)
A02 Cryptographic Failures	plain-text or SHA-1 passwords, TLS 1.0	Argon2id, TLS 1.2+, keys in a KMS (lessons 46–49)
A03 Injection	SQL injection, OS command injection, XSS	parameterised queries, output encoding
A04 Insecure Design	password reset with no rate limit	threat modeling before code (lesson 54)
A05 Security Misconfiguration	default passwords, public buckets, debug on	hardened, automated config
A06 Vulnerable and Outdated Components	library with a known CVE	dependency scanning, fast patching
A07 Identification and Authentication Failures	credential stuffing, weak sessions	MFA, rate limits, proven IdP
A08 Software and Data Integrity Failures	unsigned updates, unsafe deserialization	signatures, trusted pipelines (lesson 58)
A09 Security Logging and Monitoring Failures	breach unnoticed for months	audit logs + alerts (lesson 55)
A10 Server-Side Request Forgery (SSRF)	server fetches the metadata endpoint	allowlist destinations, block internal IPs

2 A01 Broken access control

Users act outside their permissions: reading another user's record by changing an ID, or calling an admin endpoint whose button was only hidden in the UI. It moved to #1 in 2021; 94% of applications tested had some form of it. Fix on the server: deny by default, check ownership on every request, and write tests that try other users' IDs.

3 A03 Injection

vulnerable: string building

```
q = "SELECT * FROM users WHERE name = '" + name + "'"
name = x' OR '1'='1
→ SELECT * FROM users WHERE name = 'x' OR '1'='1' (every row)
```

safe: parameterised query

```
cur.execute("SELECT * FROM users WHERE name = %s", (name,))
```

SQL text is fixed and parsed first; the value travels separately to the driver, so x' OR '1'='1 is just a strange name that matches nobody

Figure 1. Injection mixes untrusted data into code. Parameterised queries keep them apart, so input can only ever be data.

The interpreter (SQL, shell, LDAP, the browser's HTML parser) can't tell data from code, so the attacker's text runs. Cross-site scripting is the same idea in the browser: fix it with context-aware output encoding (templating engines do this by default) plus a Content Security Policy. ORMs parameterise for you, unless you build raw string queries.

4 A02, A04, A05: crypto, design, configuration

Cryptographic failures expose data: plain-text passwords, fast hashes, old TLS, hard-coded keys. **Insecure design** is a flaw in the plan, like unlimited password-reset attempts or a coupon that can be applied twice, which careful coding can't fix: threat model early. **Misconfiguration**: default credentials, open storage buckets, stack traces in error pages, directory listing, permissive CORS.

5 A10 SSRF and the rest

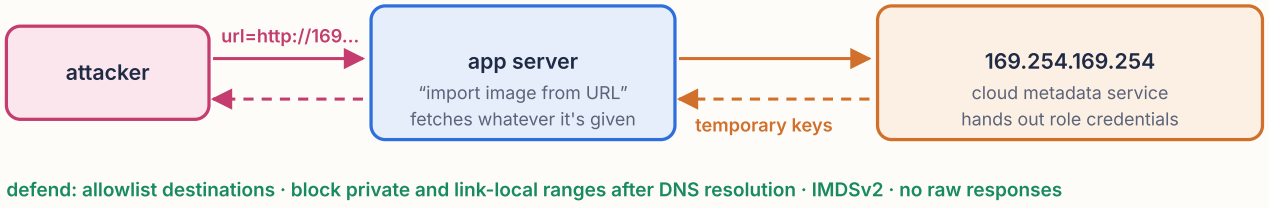


Figure 2. SSRF: the server fetches a URL the attacker chose, reaching internal services such as the cloud metadata endpoint. The 2019 Capital One breach used this path.

Vulnerable components: scan dependencies (Dependabot, Snyk, OWASP Dependency-Check) and patch continuously. The remaining categories cover authentication failures, integrity failures in software and data, and missing logging, so attacks go unnoticed. OWASP has since published a 2025 edition that keeps broken access control at #1, folds SSRF into it and adds software supply chain failures; the ideas are the same.

WORKED EXAMPLE: FIXING A SEARCH ENDPOINT

`/search?name=` built SQL by concatenation. Input `x' OR '1'='1` returned every user. The fix: `cur.execute("SELECT id, display_name FROM users WHERE name = %s", (name,))`. Also select only needed columns, run the app's DB user with read-only rights on that table, and add a test that sends quotes and `--` to every search parameter.

COMMON MISTAKES

- Escaping input by hand or with a blocklist instead of parameterising.
- Validating only in the browser; attackers call the API directly.
- Treating the list as a checklist to tick, not a guide for design reviews and tests.
- Fetching user-supplied URLs from servers with cloud credentials and no egress controls.

6 Key terms

OWASP: Open Worldwide Application Security Project

Parameterised query: fixed SQL with placeholders; values sent separately

SSRF: server tricked into requesting an attacker-chosen URL

Injection: untrusted input interpreted as code

XSS: attacker script runs in another user's browser

Insecure design: missing or flawed security control in the plan

7 Interview questions

Q1. Name the top OWASP risks and how you mitigate them. BASIC
Broken access control (server-side checks, deny by default), injection (parameterised queries, output encoding), cryptographic failures (strong hashing, modern TLS), insecure design (threat modeling), SSRF (allowlist destinations), misconfiguration and vulnerable components (hardening, dependency scanning).

Q2. What's the root cause of injection? BASIC
Mixing untrusted data with code, so the interpreter runs attacker input.

Q3. Your app lets users import a profile picture from a URL. What do you protect against? SCENARIO
SSRF: allow only http(s), resolve DNS and block private, loopback and link-local ranges (re-check after redirects), fetch from an isolated egress proxy, and enforce IMDSv2.

Q4. How do you prevent XSS in a server-rendered app? SCENARIO
Use a templating engine that auto-escapes by context, avoid raw HTML insertion, sanitise any rich text with a vetted library, and add a strict Content Security Policy.

Q5. Why can't “insecure design” be fixed by better code review? TRICKY
The control is missing from the design itself, such as no rate limit on password reset; correct code faithfully implements the flaw. It needs threat modeling and requirements.

Q6. Is input validation enough to stop SQL injection? TRICKY
No. Validation helps, but legitimate data can contain quotes (O'Brien). Parameterised queries make injection structurally impossible; validation is a second layer.

Next: SD-M-013 · OWASP API Security backendroadmap.com